Systemsoftware 2026/27 – Übung 1

Janis Elzer

Aufgabe 1

Programm: Ein Programm ist eine ausführbare Datei auf der Festplatte, also eine passive Folge von Befehlen und Daten. Es macht von selbst nichts. Erst wenn das Betriebssystem es lädt und startet, entsteht daraus ein Prozess. Dasselbe Programm kann auch mehrmals gleichzeitig laufen, dann gibt es mehrere Prozesse.

Prozess: Ein Prozess ist ein Programm in Ausführung. Er bekommt vom Betriebssystem einen eigenen, geschützten Adressraum, in dem Code und Daten des Programms liegen, und weitere Ressourcen wie geöffnete Dateien. Die eigentliche Ausführung übernehmen die Threads im Prozess. Jeder Prozess hat mindestens einen Thread.

Thread: Ein Thread ist ein Kontrollfluss innerhalb eines Prozesses. Alle Threads eines Prozesses teilen sich dessen Adressraum, haben aber jeweils einen eigenen Stack und eigene Registerinhalte (z. B. den Befehlszähler). Dadurch können sie einfach über gemeinsame Variablen zusammenarbeiten, müssen sich aber synchronisieren, damit keine Data Races entstehen.

Aufgabe 2

a)

Der Takt lässt sich nicht beliebig erhöhen, weil die Verlustleistung mit dem Takt (und quadratisch mit der Spannung) steigt. Die entstehende Abwärme kann man irgendwann nicht mehr abführen. Außerdem hilft ein höherer Takt nicht, wenn das Programm auf den Speicher, die Festplatte oder Eingaben wartet.

Mehr Kerne helfen nur beim parallelisierbaren Teil eines Programms. Der sequentielle Anteil läuft weiterhin auf einem Kern und begrenzt nach dem Amdahlschen Gesetz den Speedup auf höchstens 1/s (s = sequentieller Anteil). Dazu kommt Overhead für das Erzeugen und Synchronisieren der Threads und für gemeinsam genutzte Ressourcen wie Speicher und Festplatte. Dieser Overhead wird mit mehr Threads eher größer.

b)

Amdahlsches Gesetz:

$$T_{MultiCore}(s, n) = T_{SingleCore}(s) + T_{SingleCore}((1 - s) / n)$$

$$\text{Speedup} = T_{SingleCore} / T_{MultiCore} = 1 / (s + (1 - s) / n)$$

Rechenweg für p = 75 %, n = 4: $s = 0{,}25$. $T_{MultiCore}(0{,}25, 4) = 0{,}25\ s + 0{,}75\ s / 4 = 0{,}25\ s + 0{,}1875\ s = 0{,}4375\ s$. Speedup $= 1\ s / 0{,}4375\ s \approx 2{,}29$.

Parallelisierbarer Anteil	2 Kerne	4 Kerne	8 Kerne	16 Kerne
25 %	1,14	1,23	1,28	1,31
50 %	1,33	1,60	1,78	1,88
75 %	1,60	2,29	2,91	3,37

Aufgabe 3

Testrechner: Windows, 16 Kerne (laut Java), OpenJDK 26.0.1. Gemessen habe ich mit einem Benchmark-Programm, das nach einem Aufwärm-Lauf jede Thread-Anzahl 5-mal ausführt und den Median nimmt. Die Zeit enthält jeweils die Dateisuche und das Zählen.

3) Experimente

Threads	1	2	4	8	16	32	64
Zeit (ms)	717,6	430,3	307,1	279,9	229,6	227,7	226,4
Speedup	1,00	1,67	2,34	2,56	3,12	3,15	3,17

Beobachtungen: Mit mehr Threads wird das Programm schneller, der Speedup steigt bis 16 Threads auf ca. 3,1. Das ist aber weit weg vom idealen Faktor 16. Jede Verdopplung der Threads bringt weniger: von 1 auf 2 Threads noch Faktor 1,67, von 4 auf 8 Threads nur noch ca. 10 %. Das passt zum Amdahlschen Gesetz. Rechnet man aus dem Speedup von 3,12 bei 16 Kernen zurück, ergibt sich ein serieller Anteil von etwa 28 %. Damit wäre auch mit beliebig vielen Kernen höchstens ein Speedup von ca. 3,6 möglich. Seriell ist vor allem die Dateisuche mit Files.walk. Außerdem müssen sich alle Threads den Arbeitsspeicher und das Dateisystem teilen, sodass das Zählen nicht beliebig schneller werden kann.

Ab 16 Threads, also so vielen Threads wie Kernen, steigt der Speedup praktisch nicht mehr (32 und 64 Threads: 3,15 und 3,17). Es können nicht mehr als 16 Threads gleichzeitig laufen, die zusätzlichen Threads warten nur. Langsamer wird es dadurch aber kaum, weil der Overhead für die zusätzlichen Threads im Vergleich zur Gesamtlaufzeit klein ist.